

OPERATING SYSTEMS – FINAL PROJECT

For the 2 parts of the project you should provide all reports required and all codes, programs, network files and results

Important Note: There should be 3 assignments in each project. Each assignment report should be in the format of a research paper. Each such paper, with all tables, figures, and diagrams, should be in 10 pt, one-column, single-spaced, format, 10 pages and up to 15 pages. Min 5000 words, maximum 8000 words. These reports-papers should contain only the important material. Programming Codes, material used, including codes and papers, all relevant screenshots, and all raw results/runs of codes should be in extra appendices per assignment

FINAL PROJECT GUIDELINES

Based on the material, after these guidelines, prepare 3 research reports as analyzed next

STRUCTURE

You will produce **3 independent papers**:

Part	Paper Title Theme	Area
Paper 1	Advanced Shell Script Analysis & Optimization	Systems / Tooling (PART I)
Paper 2	Multithreaded Synchronization & Performance	Concurrency (PART II)
Paper 3	Scheduling, Queuing & System Optimization	OS Theory + Simulation (PART II)

Each paper:

- 5000–8000 words
- Contains methodology, experiments, results ✓
- Includes reproducibility & datasets ✓

PAPER 1

Title:

“Experimental Analysis, Optimization, and Design Patterns in Unix Shell Scripts for System-Level Automation”

OBJECTIVE (UPGRADED)

Instead of just explaining scripts, you will:

- Perform **systematic empirical evaluation**
- Introduce **performance metrics**
- Propose **optimized versions**
- Extract **general design patterns**

STRUCTURE

1. Introduction

- Role of shell scripting in OS automation
- Motivation: inefficiencies in legacy scripts
- Contributions:
 - Benchmark framework for shell scripts

- Optimization techniques
- Pattern classification

2. Experimental Setup

- Environment: Linux distro, Bash version
- Tools:
 - time, strace, perf, top
- Metrics:
 - Execution time
 - CPU usage
 - Memory usage
 - I/O calls

3. Script Analysis (Research Upgrade)

Use 10 only selected examples per student's team from BELOW, IN THE MATERIAL OF PART I from sections 1,2,3. Ten in total scripts from sections 1,2,3. Your report will be based on these ten scripts you have selected:

☒ mailformat

☒ rn / blank-rename

☒ encryptedpw

☒ collatz

☒ days-between

☒ Game of Life

For EACH script:

1. Functional description
2. Control/data flow diagram
3. Complexity analysis
4. Instrumentation (add logs)
5. Experimental results
6. Performance bottlenecks
7. Optimized version

4. Comparative Optimization Study

- Compare:
 - original vs optimized
- Techniques:
 - reducing subshells
 - replacing sed with built-ins
 - avoiding unnecessary pipes

5. Design Patterns in Shell Scripts

Extract patterns:

- Pipeline pattern
- Filter-transform pattern
- Iterative processing
- State-based scripting

6. Case Study: Script Refactoring

Take:

- the modular arithmetic script
- rewrite using:
 - mathematical derivation
 - direct solution (e.g Chinese Remainder Theorem insight)

7. New Script Design (Research Contribution)

Design a **novel tool**, e.g.:

- intelligent file organizer
- resource-aware log analyzer or other by adapting OPEN SOURCE tools from the Internet

Include:

- algorithm
- evaluation

8. Results & Discussion

- tables of execution times
- scalability plots

9. Conclusion

PAPER 2

Title:

“Comparative Performance Evaluation of Multithreaded Synchronization Techniques in Producer–Consumer and Dining Philosophers Problems”

OBJECTIVE

Upgrade assignment → **full experimental concurrency study**

STRUCTURE

1. Introduction

- synchronization challenges
- relevance in multicore systems

2. Related Work

- classical problems
- mutexes, semaphores, monitors

3. Methodology

Implement BOTH:

A. Producer–Consumer (Bounded Buffer)

- POSIX Threads
- Java/Python threads

B. Dining Philosophers

- multiple classical solutions:
 - monitor-based
 - semaphore-based
 - resource hierarchy

4. Experimental Design

Variables:

- N = buffer size

- M = producers
- K = consumers
- P = philosophers

Metrics:

- throughput
- latency
- context switches
- CPU usage
- deadlock frequency

5. Performance Evaluation

Produce:

- graphs:
 - time vs threads
 - CPU vs contention
- stress tests

6. Scalability Study

Generalize:

- Producer–Consumer $\rightarrow$ $M \times K$ threads
- Philosophers $\rightarrow$ N nodes

7. Comparative Analysis

Compare:

- POSIX vs Java
- locks vs monitors or semaphores vs mutexes

8. Optimization Proposal

- lock-free ideas
- adaptive buffer sizing

9. Discussion

PAPER 3

Title:

“Optimization of Multilevel Feedback Queue Scheduling and Queueing Models Using Simulation and Analytical Techniques”

OBJECTIVE

Turn scheduling exercises → **full research in OS optimization**

STRUCTURE

1. Introduction

- scheduling importance
- limitations of default RR values

2. System Model

Processes:

- burst time: random (10–1000)
- arrival: stochastic

3. MLFQ Simulation

Parameters:

- Q1, Q2 (time quantum)
- L1, L2 (time allocation)
- T (SJF vs FCFS split)

4. Metrics

- throughput
- turnaround time
- waiting time
- response time

5. Simulation Framework

- Python or Java implementation
 - Monte Carlo runs (100–500 trials)
-

6. Optimization Study

Find optimal:

- Q1, Q2
- L1, L2
- T

Use:

- grid search
- statistical averaging

7. Visualization

- heatmaps
- 2D/3D performance surfaces

8. Queuing Theory Extension

Study:

- M/M/1
- M/M/S

Evaluate:

- system stability
- queue length
- utilization

9. Randomness Study

Compare:

- uniform vs exponential vs Poisson

10. Results

11. Conclusion

✓ **Publishable Contribution:**

- empirical + analytical optimization of MLFQ
- integration with queueing theory

FINAL DELIVERABLES (UPGRADED)

Each paper must include:

- Abstract
- Keywords
- Methodology
- Results (plots + tables)
- Reproducibility notes

BONUS (FOR HIGH-LEVEL)

Add:

- GitHub repo with:
 - code
 - datasets
- reproducibility section
- comparison with literature

All reports will be based on the following **MATERIAL**

1) Running and Analyzing Scripts

Examine the following scripts. Choose only 10 per group. Run them, then shortly explain them. You should experiment with different inputs, provide the relevant results through screenshots, after extensively commenting the given codes. You should also, provide the intermediate, not only the final results of each

script, by adding “echo” commands whenever needed to show intermediate results and how each script is behaved in detail, so that to support your analysis. Therefore, the analysis of each script will not be only additional comments, in the already existing ones, but, also, step by step intermediate results proving your comments. Based on your analysis prepare a full report as above in the previous section.

Example A-1. *mailformat*: Formatting an e-mail message

```
#!/bin/bash
# mail-format.sh (ver. 1.1): Format e-mail messages.

# Gets rid of carets, tabs, and also folds excessively long lines.

# =====
#                               Standard Check for Script Argument(s)
ARGS=1
E_BADARGS=85
E_NOFILE=86

if [ $# -ne $ARGS ] # Correct number of arguments passed to script?
then
    echo "Usage: `basename $0` filename"
    exit $E_BADARGS
fi

if [ -f "$1" ]      # Check if file exists.
then
    file_name=$1
else
    echo "File \"$1\" does not exist."
    exit $E_NOFILE
fi

# -----

MAXWIDTH=70        # Width to fold excessively long lines to.

# =====
# A variable can hold a sed script.
# It's a useful technique.
sedscript='s/^>//
s/^ *>//
s/^ *//
s/          *// '
# =====

# Delete carets and tabs at beginning of lines,
#+ then fold lines to $MAXWIDTH characters.
sed "$sedscript" $1 | fold -s --width=$MAXWIDTH
                        # -s option to "fold"
                        #+ breaks lines at whitespace, if possible.

# This script was inspired by an article in a well-known trade journal
#+ extolling a 164K MS Windows utility with similar functionality.
#
# An nice set of text processing utilities and an efficient
```

```
#+ scripting language provide an alternative to the bloated executables
#+ of a clunky operating system.

exit $?
```

Example A-2. *rn*: A simple-minded file renaming utility

This script is a modification of [Example 16-22](#).

```
#!/bin/bash
# rn.sh

# Very simpleminded filename "rename" utility (based on "lowercase.sh").
#
# The "ren" utility, by Vladimir Lanin (lanin@csd2.nyu.edu),
#+ does a much better job of this.

ARGS=2
E_BADARGS=85
ONE=1                                # For getting singular/plural right (see below).

if [ $# -ne "$ARGS" ]
then
    echo "Usage: `basename $0` old-pattern new-pattern"
    # As in "rn gif jpg", which renames all gif files in working directory to
    # jpg.
    exit $E_BADARGS
fi

number=0                             # Keeps track of how many files actually renamed.

for filename in *$1*                 # Traverse all matching files in directory.
do
    if [ -f "$filename" ]           # If finds match...
    then
        fname=`basename $filename`  # Strip off path.
        n=`echo $fname | sed -e "s/$1/$2/"` # Substitute new for old in
        filename.                  #
        mv $fname $n               # Rename.
        let "number += 1"
    fi
done

if [ "$number" -eq "$ONE" ]          # For correct grammar.
then
    echo "$number file renamed."
else
    echo "$number files renamed."
fi

exit $?
```

```
# Exercises:
# -----
# What types of files will this not work on?
# How can this be fixed?
```

Example A-3. *blank-rename*: Renames filenames containing blanks

This is an even simpler-minded version of previous script.

```
#!/bin/bash
# blank-rename.sh
#
# Substitutes underscores for blanks in all the filenames in a directory.

ONE=1                                # For getting singular/plural right (see below).
number=0                             # Keeps track of how many files actually renamed.
FOUND=0                              # Successful return value.

for filename in *                    # Traverse all files in directory.
do
    echo "$filename" | grep -q " "    # Check whether filename
    if [ $? -eq $FOUND ]             #+ contains space(s).
    then
        fname=$filename              # Yes, this filename needs work.
        n=`echo $fname | sed -e "s/ /_/g"` # Substitute underscore for
blank.                                #
        mv "$fname" "$n"              # Do the actual renaming.
        let "number += 1"
    fi
done

if [ "$number" -eq "$ONE" ]           # For correct grammar.
then
    echo "$number file renamed."
else
    echo "$number files renamed."
fi

exit 0
```

Example A-4. *encryptedpw*: Uploading to an ftp site, using a locally encrypted password

```
#!/bin/bash

# Example "ex72.sh" modified to use encrypted password.

# Note that this is still rather insecure,
#+ since the decrypted password is sent in the clear.
# Use something like "ssh" if this is a concern.

E_BADARGS=85
```

```

if [ -z "$1" ]
then
    echo "Usage: `basename $0` filename"
    exit $E_BADARGS
fi

Username=bozo          # Change to suit.
pword=/home/bozo/secret/password_encrypted.file
# File containing encrypted password.

Filename=`basename $1` # Strips pathname out of file name.

Server="XXX"
Directory="YYY"        # Change above to actual server name & directory.

Password=`cruft <$pword` # Decrypt password.
# Uses the author's own "cruft" file encryption package,
#+ based on the classic "onetime pad" algorithm,
#+ and obtainable from:
#+ Primary-site:  ftp://ibiblio.org/pub/Linux/utils/file
#+               cruft-0.2.tar.gz [16k]

ftp -n $Server <<End-Of-Session
user $Username $Password
binary
bell
cd $Directory
put $Filename
bye
End-Of-Session
# -n option to "ftp" disables auto-login.
# Note that "bell" rings 'bell' after each file transfer.

exit 0

```

Example A-5. *copy-cd*: Copying a data CD

```

#!/bin/bash
# copy-cd.sh: copying a data CD

CDROM=/dev/cdrom          # CD ROM device
OF=/home/bozo/projects/cdimage.iso # output file
#           /xxxx/xxxxxxxxx/      Change to suit your system.
BLOCKSIZE=2048
# SPEED=10                  # If unspecified, uses max spd.
# DEVICE=/dev/cdrom         older version.
DEVICE="1,0,0"

echo; echo "Insert source CD, but do *not* mount it."
echo "Press ENTER when ready. "
read ready                # Wait for input, $ready not used.

```

```

echo; echo "Copying the source CD to $OF."
echo "This may take a while. Please be patient."

dd if=$CDROM of=$OF bs=$BLOCKSIZE          # Raw device copy.

echo; echo "Remove data CD."
echo "Insert blank CDR."
echo "Press ENTER when ready. "
read ready                                # Wait for input, $ready not used.

echo "Copying $OF to CDR."

# cdrecord -v -isozsize speed=$SPEED dev=$DEVICE $OF    # Old version.
wodim -v -isozsize dev=$DEVICE $OF
# Uses Joerg Schilling's "cdrecord" package (see its docs).
# http://www.fokus.gmd.de/nthp/employees/schilling/cdrecord.html
# Newer Linux distros may use "wodim" rather than "cdrecord" ...

echo; echo "Done copying $OF to CDR on device $CDROM."

echo "Do you want to erase the image file (y/n)? " # Probably a huge file.
read answer

case "$answer" in
[yY]) rm -f $OF
      echo "$OF erased."
      ;;
*)    echo "$OF not erased.>";;
esac

echo

# Exercise:
# Change the above "case" statement to also accept "yes" and "Yes" as input.

exit 0

```

Example A-6. Collatz series

```

#!/bin/bash
# collatz.sh

# The notorious "hailstone" or Collatz series.
# -----
# 1) Get the integer "seed" from the command-line.
# 2) NUMBER <-- seed
# 3) Print NUMBER.
# 4) If NUMBER is even, divide by 2, or
# 5)+ if odd, multiply by 3 and add 1.
# 6) NUMBER <-- result
# 7) Loop back to step 3 (for specified number of iterations).
#
# The theory is that every such sequence,

```

```

#+ no matter how large the initial value,
#+ eventually settles down to repeating "4,2,1..." cycles,
#+ even after fluctuating through a wide range of values.
#
# This is an instance of an "iterate,"
#+ an operation that feeds its output back into its input.
# Sometimes the result is a "chaotic" series.

MAX_ITERATIONS=200
# For large seed numbers (>32000), try increasing MAX_ITERATIONS.

h=${1:-$$}          # Seed.
                    # Use $PID as seed,
                    #+ if not specified as command-line arg.

echo
echo "C($h) -*- $MAX_ITERATIONS Iterations"
echo

for ((i=1; i<=MAX_ITERATIONS; i++))
do

# echo -n "$h  "
#          ^^^
#          tab
# printf does it better ...
COLWIDTH=%7d
printf $COLWIDTH $h

    let "remainder = h % 2"
    if [ "$remainder" -eq 0 ]    # Even?
    then
        let "h /= 2"          # Divide by 2.
    else
        let "h = h*3 + 1"      # Multiply by 3 and add 1.
    fi

COLUMNS=10          # Output 10 values per line.
let "line_break = i % $COLUMNS"
if [ "$line_break" -eq 0 ]
then
    echo
fi

done

echo

# For more information on this strange mathematical function,
#+ see Computers, Pattern, Chaos, and Beauty, by Pickover, p. 185 ff.,
#+ as listed in the bibliography.

exit 0

```

Example A-7. *days-between*: Days between two dates

```
#!/bin/bash
# days-between.sh:      Number of days between two dates.
# Usage: ./days-between.sh [M]M/[D]D/YYYY [M]M/[D]D/YYYY
#
# Note: Script modified to account for changes in Bash, v. 2.05b +,
#+      that closed the loophole permitting large negative
#+      integer return values.

ARGS=2                # Two command-line parameters expected.
E_PARAM_ERR=85        # Param error.

REFYR=1600            # Reference year.
CENTURY=100
DIY=365
ADJ_DIY=367           # Adjusted for leap year + fraction.
MIY=12
DIM=31
LEAPCYCLE=4

MAXRETVAL=255         # Largest permissible
                      #+ positive return value from a function.

diff=                 # Declare global variable for date difference.
value=                # Declare global variable for absolute value.
day=                  # Declare globals for day, month, year.
month=
year=

Param_Error ()        # Command-line parameters wrong.
{
    echo "Usage: `basename $0` [M]M/[D]D/YYYY [M]M/[D]D/YYYY"
    echo "          (date must be after 1/3/1600)"
    exit $E_PARAM_ERR
}

Parse_Date ()          # Parse date from command-line params.
{
    month=${1%/**}
    dm=${1%/**}        # Day and month.
    day=${dm#*/}
    let "year = `basename $1`" # Not a filename, but works just the same.
}

check_date ()          # Checks for invalid date(s) passed.
{
    [ "$day" -gt "$DIM" ] || [ "$month" -gt "$MIY" ] ||
    [ "$year" -lt "$REFYR" ] && Param_Error
    # Exit script on bad value(s).
    # Uses or-list / and-list.
    #
    # Exercise: Implement more rigorous date checking.
}
```



```

}

strip_leading_zero () # Better to strip possible leading zero(s)
{
    return ${1#0}      #+ from day and/or month
}                    #+ since otherwise Bash will interpret them
                    #+ as octal values (POSIX.2, sect 2.9.2.1).

day_index ()          # Gauss' Formula:
{
    # Days from March 1, 1600 to date passed as param.
    #          ^^^^^^^^^^^^^^^
    day=$1
    month=$2
    year=$3

    let "month = $month - 2"
    if [ "$month" -le 0 ]
    then
        let "month += 12"
        let "year -= 1"
    fi

    let "year -= $REFYR"
    let "indexyr = $year / $CENTURY"

    let "Days = $DIY*$year + $year/$LEAPCYCLE - $indexyr \
        + $indexyr/$LEAPCYCLE + $ADJ_DIY*$month/$MIY + $day - $DIM"
    # For an in-depth explanation of this algorithm, see
    #+ http://weblogs.asp.net/pgreborio/archive/2005/01/06/347968.aspx

    echo $Days
}

calculate_difference () # Difference between two day indices.
{
    let "diff = $1 - $2" # Global variable.
}

abs () # Absolute value
{
    # Uses global "value" variable.
    if [ "$1" -lt 0 ] # If negative
    then #+ then
        let "value = 0 - $1" #+ change sign,
    else #+ else
        let "value = $1" #+ leave it alone.
    fi
}

```

```

if [ $# -ne "$ARGS" ]          # Require two command-line params.
then
    Param_Error
fi

Parse_Date $1
check_date $day $month $year    # See if valid date.

strip_leading_zero $day         # Remove any leading zeroes
day=$?                          #+ on day and/or month.
strip_leading_zero $month
month=$?

let "date1 = `day_index $day $month $year`"

Parse_Date $2
check_date $day $month $year

strip_leading_zero $day
day=$?
strip_leading_zero $month
month=$?

date2=$(day_index $day $month $year) # Command substitution.

calculate_difference $date1 $date2

abs $diff                      # Make sure it's positive.
diff=$value

echo $diff

exit 0

# Exercise:
# -----
# If given only one command-line parameter, have the script
#+ use today's date as the second.

# Compare this script with
#+ the implementation of Gauss' Formula in a C program at
#+ http://buschencrew.hypermart.net/software/datedif

```

Example A-8. Making a *dictionary*

```

#!/bin/bash
# makedict.sh [make dictionary]

# Modification of /usr/sbin/mkdict (/usr/sbin/cracklib-forman) script.
# Original script copyright 1993, by Alec Muffett.
#
# This modified script included in this document in a manner

```

```

#+ consistent with the "LICENSE" document of the "Crack" package
#+ that the original script is a part of.

# This script processes text files to produce a sorted list
#+ of words found in the files.
# This may be useful for compiling dictionaries
#+ and for other lexicographic purposes.

E_BADARGS=85

if [ ! -r "$1" ]          # Need at least one
then                      #+ valid file argument.
    echo "Usage: $0 files-to-process"
    exit $E_BADARGS
fi

# SORT="sort"            # No longer necessary to define
                        #+ options to sort. Changed from
                        #+ original script.

cat $* |                 # Dump specified files to stdout.
    tr A-Z a-z |         # Convert to lowercase.
    tr ' ' '\012' |     # New: change spaces to newlines.
#    tr -cd '\012[a-z][0-9]' | # Get rid of everything
                        #+ non-alphanumeric (in orig. script).
    tr -c '\012a-z' '\012' | # Rather than deleting non-alpha
                        #+ chars, change them to newlines.
    sort |               # $SORT options unnecessary now.
    uniq |               # Remove duplicates.
    grep -v '^#' |      # Delete lines starting with #.
    grep -v '^$'        # Delete blank lines.

exit $?

```

Example A-9. Soundex conversion

```

#!/bin/bash
# soundex.sh: Calculate "soundex" code for names

# =====
#         Soundex script
#         by
#         Mendel Cooper
#         thegrendel.abs@gmail.com
#         reldate: 23 January, 2002
#
# Placed in the Public Domain.
#
# A slightly different version of this script appeared in
#+ Ed Schaefer's July, 2002 "Shell Corner" column
#+ in "Unix Review" on-line,
#+ http://www.unixreview.com/documents/unil026336632258/
# =====

```

```

ARGCOUNT=1                                # Need name as argument.
E_WRONGARGS=90

if [ $# -ne "$ARGCOUNT" ]
then
    echo "Usage: `basename $0` name"
    exit $E_WRONGARGS
fi

assign_value ()                             # Assigns numerical value
{                                           #+ to letters of name.

    val1=bfpv                               # 'b,f,p,v' = 1
    val2=cgjkqsxz                           # 'c,g,j,k,q,s,x,z' = 2
    val3=dt                                  # etc.
    val4=l
    val5=mn
    val6=r

# Exceptionally clever use of 'tr' follows.
# Try to figure out what is going on here.

value=$( echo "$1" \
| tr -d wh \
| tr $val1 1 | tr $val2 2 | tr $val3 3 \
| tr $val4 4 | tr $val5 5 | tr $val6 6 \
| tr -s 123456 \
| tr -d aeiouy )

# Assign letter values.
# Remove duplicate numbers, except when separated by vowels.
# Ignore vowels, except as separators, so delete them last.
# Ignore 'w' and 'h', even as separators, so delete them first.
#
# The above command substitution lays more pipe than a plumber <g>.

}

input_name="$1"
echo
echo "Name = $input_name"

# Change all characters of name input to lowercase.
# -----
name=$( echo $input_name | tr A-Z a-z )
# -----
# Just in case argument to script is mixed case.

# Prefix of soundex code: first letter of name.
# -----

```

```

char_pos=0                                # Initialize character position.
prefix0=${name:$char_pos:1}
prefix=`echo $prefix0 | tr a-z A-Z`
                                           # Uppercase 1st letter of soundex.

let "char_pos += 1"                        # Bump character position to 2nd letter of
name.
name1=${name:$char_pos}

# ++++++ Exception Patch ++++++
# Now, we run both the input name and the name shifted one char
#+ to the right through the value-assigning function.
# If we get the same value out, that means that the first two characters
#+ of the name have the same value assigned, and that one should cancel.
# However, we also need to test whether the first letter of the name
#+ is a vowel or 'w' or 'h', because otherwise this would bollix things up.

char1=`echo $prefix | tr A-Z a-z`         # First letter of name, lowercased.

assign_value $name
s1=$value
assign_value $name1
s2=$value
assign_value $char1
s3=$value
s3=9$s3                                   # If first letter of name is a vowel
                                           #+ or 'w' or 'h',
                                           #+ then its "value" will be null

(unset).                                  #+ Therefore, set it to 9, an otherwise
                                           #+ unused value, which can be tested for.

if [[ "$s1" -ne "$s2" || "$s3" -eq 9 ]]
then
    suffix=$s2
else
    suffix=${s2:$char_pos}
fi
# ++++++ end Exception Patch ++++++

padding=000                              # Use at most 3 zeroes to pad.

soun=$prefix$suffix$padding               # Pad with zeroes.

MAXLEN=4                                  # Truncate to maximum of 4 chars.
soundex=${soun:0:$MAXLEN}

echo "Soundex = $soundex"

echo

```

```
# The soundex code is a method of indexing and classifying names
#+ by grouping together the ones that sound alike.
# The soundex code for a given name is the first letter of the name,
#+ followed by a calculated three-number code.
# Similar sounding names should have almost the same soundex codes.

# Examples:
# Smith and Smythe both have a "S-530" soundex.
# Harrison = H-625
# Hargison = H-622
# Harriman = H-655

# This works out fairly well in practice, but there are numerous anomalies.
#
# The U.S. Census and certain other governmental agencies use soundex,
# as do genealogical researchers.
#
# For more information,
#+ see the "National Archives and Records Administration home page",
#+ http://www.nara.gov/genealogy/soundex/soundex.html

# Exercise:
# -----
# Simplify the "Exception Patch" section of this script.

exit 0
```

[illegible]

```

#           4*5      The * is the cell under consideration. #
#           678                                           #
#                                                         #
# 2) A living cell with either 2 or 3 living neighbors remains alive. #
SURVIVE=2                                           #
# 3) A dead cell with 3 living neighbors comes alive, a "birth." #
BIRTH=3                                           #
# 4) All other cases result in a dead cell for the next generation. #
# ##### #
# ##### #

startfile=gen0    # Read the starting generation from the file "gen0" ...
                  # Default, if no other file specified when invoking script.
                  #
if [ -n "$1" ]    # Specify another "generation 0" file.
then
    startfile="$1"
fi

#####
# Abort script if "startfile" not specified
#+ and
#+ default file "gen0" not present.

E_NOSTARTFILE=86

if [ ! -e "$startfile" ]
then
    echo "Startfile \"'$startfile'\" missing!"
    exit $E_NOSTARTFILE
fi
#####

ALIVE1=.
DEAD1=_
                # Represent living and dead cells in the start-up file.

# -----#
# This script uses a 10 x 10 grid (may be increased,
#+ but a large grid will slow down execution).
ROWS=10
COLS=10
# Change above two variables to match desired grid size.
# -----#

GENERATIONS=10    # How many generations to cycle through.
                  # Adjust this upwards
                  #+ if you have time on your hands.

NONE_ALIVE=85     # Exit status on premature bailout,
                  #+ if no cells left alive.

DELAY=2           # Pause between generations.
TRUE=0
FALSE=1
ALIVE=0

```

[illegible]


```

    if [ -z "$1" -o -z "$2" ]          # Mandatory arguments missing?
    then
        return $FALSE
    fi

local row
local lower_limit=0                    # Disallow negative coordinate.
local upper_limit
local left
local right

let "upper_limit = $ROWS * $COLS - 1" # Total number of cells.

if [ "$1" -lt "$lower_limit" -o "$1" -gt "$upper_limit" ]
then
    return $FALSE                      # Out of array bounds.
fi

row=$2
let "left = $row * $COLS"              # Left limit.
let "right = $left + $COLS - 1"        # Right limit.

if [ "$1" -lt "$left" -o "$1" -gt "$right" ]
then
    return $FALSE                      # Beyond row boundary.
fi

return $TRUE                          # Valid coordinate.
}

IsAlive ()                            # Test whether cell is alive.
# Takes array, cell number, and
{                                     #+ state of cell as arguments.
    GetCount "$1" $2                 # Get alive cell count in neighborhood.
    local nhbd=$?

    if [ "$nhbd" -eq "$BIRTH" ]      # Alive in any case.
    then
        return $ALIVE
    fi

    if [ "$3" = "." -a "$nhbd" -eq "$SURVIVE" ]
    then
        return $ALIVE                # Alive only if previously alive.
    fi

    return $DEAD                      # Defaults to dead.
}

GetCount ()                           # Count live cells in passed cell's neighborhood.
# Two arguments needed:

```

```

# $1) variable holding array
# $2) cell number
{
  local cell_number=$2
  local array
  local top
  local center
  local bottom
  local r
  local row
  local i
  local t_top
  local t_cen
  local t_bot
  local count=0
  local ROW_NHBD=3

  array=( `echo "$1" ` )

  let "top = $cell_number - $COLS - 1"      # Set up cell neighborhood.
  let "center = $cell_number - 1"
  let "bottom = $cell_number + $COLS - 1"
  let "r = $cell_number / $COLS"

  for ((i=0; i<$ROW_NHBD; i++))             # Traverse from left to right.
  do
    let "t_top = $top + $i"
    let "t_cen = $center + $i"
    let "t_bot = $bottom + $i"

    let "row = $r"                          # Count center row.
    IsValid $t_cen $row                     # Valid cell position?
    if [ $? -eq "$TRUE" ]
    then
      if [ ${array[$t_cen]} = "$ALIVE1" ] # Is it alive?
      then                                # If yes, then ...
        let "count += 1"                 # Increment count.
      fi
    fi

    let "row = $r - 1"                      # Count top row.
    IsValid $t_top $row
    if [ $? -eq "$TRUE" ]
    then
      if [ ${array[$t_top]} = "$ALIVE1" ] # Redundancy here.
      then                                # Can it be optimized?
        let "count += 1"
      fi
    fi

    let "row = $r + 1"                      # Count bottom row.
    IsValid $t_bot $row
    if [ $? -eq "$TRUE" ]
    then
      if [ ${array[$t_bot]} = "$ALIVE1" ]

```

```

        then
            let "count += 1"
        fi
    fi

done

if [ ${array[$cell_number]} = "$ALIVE1" ]
then
    let "count -= 1"          # Make sure value of tested cell itself
    fi                      #+ is not counted.

return $count
}

next_gen ()                # Update generation array.
{

local array
local i=0

array=( `echo "$1"` )      # Convert passed arg to array.

while [ "$i" -lt "$cells" ]
do
    IsAlive "$1" $i ${array[$i]} # Is the cell alive?
    if [ $? -eq "$ALIVE" ]
    then
        array[$i]=.           # If alive, then
                                #+ represent the cell as a period.
    else
        array[$i]="_"         # Otherwise underscore
                                #+ (will later be converted to space).
    fi
    let "i += 1"
done

#   let "generation += 1"      # Increment generation count.
### Why was the above line commented out?

# Set variable to pass as parameter to "display" function.
avar=`echo ${array[@]}`      # Convert array back to string variable.
display "$avar"              # Display it.
echo; echo
echo "Generation $generation - $alive alive"

if [ "$alive" -eq 0 ]
then
    echo
    echo "Premature exit: no more cells alive!"
    exit $NONE_ALIVE          # No point in continuing
    fi                      #+ if no live cells.

```

```

}

# =====

# main ()
# {

# Load initial array with contents of startup file.
initial=( `cat "$startfile" | sed -e '/#/d' | tr -d '\n' | \
# Delete lines containing '#' comment character.
sed -e 's/\./\./g' -e 's/_/_/g'` )
# Remove linefeeds and insert space between elements.

clear          # Clear screen.

echo #          Title
setterm -reverse on
echo "======"
setterm -reverse off
echo "    $GENERATIONS generations"
echo "          of"
echo "\"Life in the Slow Lane\""
setterm -reverse on
echo "======"
setterm -reverse off

sleep $DELAY    # Display "splash screen" for 2 seconds.

# ----- Display first generation. -----
Gen0=`echo ${initial[@]}`
display "$Gen0"          # Display only.
echo; echo
echo "Generation $generation - $alive alive"
sleep $DELAY
# -----

let "generation += 1"      # Bump generation count.
echo

# ----- Display second generation. -----
Cur=`echo ${initial[@]}`
next_gen "$Cur"          # Update & display.
sleep $DELAY
# -----

let "generation += 1"      # Increment generation count.

# ----- Main loop for displaying subsequent generations -----
while [ "$generation" -le "$GENERATIONS" ]
do
    Cur="$savar"
    next_gen "$Cur"
    let "generation += 1"

```

```

    sleep $DELAY
done
# =====

echo
# }

exit 0    # EOF:EOF

# The grid in this script has a "boundary problem."
# The the top, bottom, and sides border on a void of dead cells.
# Exercise: Change the script to have the grid wrap around,
# +           so that the left and right sides will "touch,"
# +           as will the top and bottom.
#
# Exercise: Create a new "gen0" file to seed this script.
#           Use a 12 x 16 grid, instead of the original 10 x 10 one.
#           Make the necessary changes to the script,
#+          so it will run with the altered file.
#
# Exercise: Modify this script so that it can determine the grid size
#+          from the "gen0" file, and set any variables necessary
#+          for the script to run.
#           This would make unnecessary any changes to variables
#+          in the script for an altered grid size.
#
# Exercise: Optimize this script.
#           It has redundant code.

```

Example A-11. Data file for *Game of Life*

```

# gen0
#
# This is an example "generation 0" start-up file for "life.sh".
# -----
# The "gen0" file is a 10 x 10 grid using a period (.) for live cells,
#+ and an underscore (_) for dead ones. We cannot simply use spaces
#+ for dead cells in this file because of a peculiarity in Bash arrays.
# [Exercise for the reader: explain this.]
#
# Lines beginning with a '#' are comments, and the script ignores them.
__._..__
_._._.
_____.
_.._____
_____.
..__...
_____.
_____.
_._._.
_.._.
_.._.

```

+++

The following script is by Mark Moraes of the University of Toronto. See the file `Moraes-COPYRIGHT` for permissions and restrictions. This file is included in the combined [HTML/source tarball](#) of the *ABS Guide*.

Example A-12. *behead*: Removing mail and news message headers

```
#!/bin/sh
# Strips off the header from a mail/News message i.e. till the first
#+ empty line.
# Author: Mark Moraes, University of Toronto

# ==> These comments added by author of this document.

if [ $# -eq 0 ]; then
# ==> If no command-line args present, then works on file redirected to
stdin.
    sed -e '1,/^\$/d' -e '/^[ \t]*$/d'
    # --> Delete empty lines and all lines until
    # --> first one beginning with white space.
else
# ==> If command-line args present, then work on files named.
    for i do
        sed -e '1,/^\$/d' -e '/^[ \t]*$/d' $i
        # --> Ditto, as above.
    done
fi

exit

# ==> Exercise: Add error checking and other options.
# ==>
# ==> Note that the small sed script repeats, except for the arg passed.
# ==> Does it make sense to embed it in a function? Why or why not?

/*
 * Copyright University of Toronto 1988, 1989.
 * Written by Mark Moraes
 *
 * Permission is granted to anyone to use this software for any purpose on
 * any computer system, and to alter it and redistribute it freely, subject
 * to the following restrictions:
 *
 * 1. The author and the University of Toronto are not responsible
 *    for the consequences of use of this software, no matter how awful,
 *    even if they arise from flaws in it.
 *
 * 2. The origin of this software must not be misrepresented, either by
 *    explicit claim or by omission. Since few users ever read sources,
 *    credits must appear in the documentation.
 *
 * 3. Altered versions must be plainly marked as such, and must not be
```

```
*      misrepresented as being the original software.  Since few users
*      ever read sources, credits must appear in the documentation.
*
* 4. This notice may not be removed or altered.
*/
```

+

Antek Sawicki contributed the following script, which makes very clever use of the parameter substitution operators discussed in [Section 10.2](#).

Example A-13. *password*: Generating random 8-character passwords

```
#!/bin/bash
#
#
# Random password generator for Bash 2.x +
#+ by Antek Sawicki <tenox@tenox.tc>,
#+ who generously gave usage permission to the ABS Guide author.
#
# ==> Comments added by document author ==>

MATRIX="0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
# ==> Password will consist of alphanumeric characters.
LENGTH="8"
# ==> May change 'LENGTH' for longer password.

while [ "${n:=1}" -le "$LENGTH" ]
# ==> Recall that := is "default substitution" operator.
# ==> So, if 'n' has not been initialized, set it to 1.
do
    PASS="$PASS${MATRIX:${RANDOM%${#MATRIX}}:1}"
    # ==> Very clever, but tricky.

    # ==> Starting from the innermost nesting...
    # ==> ${#MATRIX} returns length of array MATRIX.

    # ==> $RANDOM%${#MATRIX} returns random number between 1
    # ==> and [length of MATRIX] - 1.

    # ==> ${MATRIX:${RANDOM%${#MATRIX}}:1}
    # ==> returns expansion of MATRIX at random position, by length 1.
    # ==> See {var:pos:len} parameter substitution in Chapter 9.
    # ==> and the associated examples.

    # ==> PASS=... simply pastes this result onto previous PASS
    (concatenation).

    # ==> To visualize this more clearly, uncomment the following line
    #
    #         echo "$PASS"
    # ==> to see PASS being built up,
```

```

        # ==> one character at a time, each iteration of the loop.

        let n+=1
        # ==> Increment 'n' for next pass.
done

echo "$PASS"          # ==> Or, redirect to a file, as desired.

exit 0

```

+

James R. Van Zandt contributed this script which uses named pipes and, in his words, "really exercises quoting and escaping."

Example A-14. *fifo*: Making daily backups, using named pipes

```

#!/bin/bash
# ==> Script by James R. Van Zandt, and used here with his permission.

# ==> Comments added by author of this document.

HERE=`uname -n`      # ==> hostname
THERE=bilbo
echo "starting remote backup to $THERE at `date +%r`"
# ==> `date +%r` returns time in 12-hour format, i.e. "08:08:34 PM".

# make sure /pipe really is a pipe and not a plain file
rm -rf /pipe
mkfifo /pipe        # ==> Create a "named pipe", named "/pipe" ...

# ==> 'su xyz' runs commands as user "xyz".
# ==> 'ssh' invokes secure shell (remote login client).
su xyz -c "ssh $THERE \"cat > /home/xyz/backup/${HERE}-daily.tar.gz\" <
/pipe"&
cd /
tar -czf - bin boot dev etc home info lib man root sbin share usr var >
/pipe
# ==> Uses named pipe, /pipe, to communicate between processes:
# ==> 'tar/gzip' writes to /pipe and 'ssh' reads from /pipe.

# ==> The end result is this backs up the main directories, from / on down.

# ==> What are the advantages of a "named pipe" in this situation,
# ==>+ as opposed to an "anonymous pipe", with |?
# ==> Will an anonymous pipe even work here?

# ==> Is it necessary to delete the pipe before exiting the script?
# ==> How could that be done?

exit 0

```


+

Stéphane Chazelas used the following script to demonstrate generating prime numbers without arrays.

Example A-15. Generating prime numbers using the modulo operator

```
#!/bin/bash
# primes.sh: Generate prime numbers, without using arrays.
# Script contributed by Stephane Chazelas.

# This does *not* use the classic "Sieve of Eratosthenes" algorithm,
#+ but instead the more intuitive method of testing each candidate number
#+ for factors (divisors), using the "%" modulo operator.

LIMIT=1000                                # Primes, 2 ... 1000.

Primes()
{
    (( n = $1 + 1 ))                        # Bump to next integer.
    shift                                  # Next parameter in list.
    # echo "_n=$n i=$i_"

    if (( n == LIMIT ))
    then echo $*
    return
    fi

    for i; do
        # echo "-n=$n i=$i-"
        (( i * i > n )) && break             # Optimization.
        (( n % i )) && continue             # Sift out non-primes using modulo operator.
        Primes $n $@                       # Recursion inside loop.
        return
    done

    Primes $n $@ $n                        # Recursion outside loop.
                                           # Successively accumulate
                                           # positional parameters.
                                           # "$@" is the accumulating list of primes.
}

Primes 1

exit $?

# Pipe output of the script to 'fmt' for prettier printing.

# Uncomment lines 16 and 24 to help figure out what is going on.

# Compare the speed of this algorithm for generating primes
#+ with the Sieve of Eratosthenes (ex68.sh).
```

```
# Exercise: Rewrite this script without recursion.
```

+

Rick Boivie's revision of Jordi Sanfeliu's *tree* script.

Example A-16. *tree*: Displaying a directory tree

```
#!/bin/bash
# tree.sh

# Written by Rick Boivie.
# Used with permission.
# This is a revised and simplified version of a script
#+ by Jordi Sanfeliu (the original author), and patched by Ian Kjos.
# This script replaces the earlier version used in
#+ previous releases of the Advanced Bash Scripting Guide.
# Copyright (c) 2002, by Jordi Sanfeliu, Rick Boivie, and Ian Kjos.

# ==> Comments added by the author of this document.

search () {
for dir in `echo *`
# ==> `echo *` lists all the files in current working directory,
#+ ==> without line breaks.
# ==> Similar effect to for dir in *
# ==> but "dir in `echo *`" will not handle filenames with blanks.
do
if [ -d "$dir" ] ; then # ==> If it is a directory (-d)...
zz=0                  # ==> Temp variable, keeping track of
                        # directory level.
while [ $zz != $1 ]   # Keep track of inner nested loop.
do
echo -n "| "          # ==> Display vertical connector symbol,
                        # ==> with 2 spaces & no line feed
                        # in order to indent.
zz=`expr $zz + 1`     # ==> Increment zz.
done

if [ -L "$dir" ] ; then # ==> If directory is a symbolic link...
echo "+---$dir" `ls -l $dir | sed 's/^.*'$dir' //'`
# ==> Display horiz. connector and list directory name, but...
# ==> delete date/time part of long listing.
else
echo "+---$dir"        # ==> Display horizontal connector symbol...
# ==> and print directory name.
numdirs=`expr $numdirs + 1` # ==> Increment directory count.
if cd "$dir" ; then        # ==> If can move to subdirectory...
search `expr $1 + 1`      # with recursion ;- )
# ==> Function calls itself.
cd ..
fi
fi
fi
```

```

        fi
    fi
done
}

if [ $# != 0 ] ; then
    cd $1    # Move to indicated directory.
#else    # stay in current directory
fi

echo "Initial directory = `pwd`"
numdirs=0

search 0
echo "Total directories = $numdirs"

exit 0

```

Patsie's version of a directory *tree* script.

Example A-17. *tree2*: Alternate directory tree script

```

#!/bin/bash
# tree2.sh

# Lightly modified/reformatted by ABS Guide author.
# Included in ABS Guide with permission of script author (thanks!).

## Recursive file/dirsizes checking script, by Patsie
##
## This script builds a list of files/directories and their size (du -akx)
## and processes this list to a human readable tree shape
## The 'du -akx' is only as good as the permissions the owner has.
## So preferably run as root* to get the best results, or use only on
## directories for which you have read permissions. Anything you can't
## read is not in the list.

#* ABS Guide author advises caution when running scripts as root!

##### THIS IS CONFIGURABLE #####

TOP=5                # Top 5 biggest (sub)directories.
MAXRECURS=5          # Max 5 subdirectories/recursions deep.
E_BL=80              # Blank line already returned.
E_DIR=81             # Directory not specified.

##### DON'T CHANGE ANYTHING BELOW THIS LINE #####

PID=$$               # Our own process ID.
SELF=`basename $0`   # Our own program name.
TMP="/tmp/${SELF}.${PID}.tmp" # Temporary 'du' result.

```

```

# Convert number to dotted thousand.
function dot { echo "          $" |
                sed -e :a -e 's/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/;ta' |
                tail -c 12; }

# Usage: tree <recursion> <indent prefix> <min size> <directory>
function tree {
    recurs="$1"          # How deep nested are we?
    prefix="$2"          # What do we display before file/dirname?
    minsize="$3"         # What is the minimum file/dirsizes?
    dirname="$4"         # Which directory are we checking?

# Get ($TOP) biggest subdirs/subfiles from TMP file.
    LIST=`egrep "[[:space:]]${dirname}/[^/]*$" "$TMP" |
          awk '{if($1>'$minsize') print;}' | sort -nr | head -$TOP`
    [ -z "$LIST" ] && return          # Empty list, then go back.

    cnt=0
    num=`echo "$LIST" | wc -l`       # How many entries in the list.

## Main loop
    echo "$LIST" | while read size name; do
        ((cnt+=1))                # Count entry number.
        bname=`basename "$name"`  # We only need a basename of the entry.
        [ -d "$name" ] && bname="$bname/"
                                   # If it's a directory, append a slash.
        echo "`dot $size`$prefix +-$bname"
                                   # Display the result.
        # Call ourself recursively if it's a directory
        #+ and we're not nested too deep ($MAXRECURS).
        # The recursion goes up: $((recurs+1))
        # The prefix gets a space if it's the last entry,
        #+ or a pipe if there are more entries.
        # The minimum file/dirsizes becomes
        #+ a tenth of his parent: $((size/10)).
        # Last argument is the full directory name to check.
        if [ -d "$name" -a $recurs -lt $MAXRECURS ]; then
            [ $cnt -lt $num ] \
                || (tree $((recurs+1)) "$prefix " $((size/10)) "$name") \
                && (tree $((recurs+1)) "$prefix|" $((size/10)) "$name")
        fi
    done

    [ $? -eq 0 ] && echo "          $prefix"
    # Every time we jump back add a 'blank' line.
    return $E_BL
    # We return 80 to tell we added a blank line already.
}

###          ###
### main program ###
###          ###

rootdir="$@"
[ -d "$rootdir" ] ||
{ echo "$SELF: Usage: $SELF <directory>" >&2; exit $E_DIR; }

```

```

# We should be called with a directory name.

echo "Building inventory list, please wait ..."
# Show "please wait" message.
du -akx "$rootdir" 1>"$TMP" 2>/dev/null
# Build a temporary list of all files/dirs and their size.
size=`tail -1 "$TMP" | awk '{print $1}'`
# What is our rootdirectory's size?
echo "`dot $size` $rootdir"
# Display rootdirectory's entry.
tree 0 "" 0 "$rootdir"
# Display the tree below our rootdirectory.

rm "$TMP" 2>/dev/null
# Clean up TMP file.

exit $?

```

Noah Friedman permitted use of his *string function* script. It essentially reproduces some of the C-library string manipulation functions.

Example A-18. *string functions*: C-style string functions

```

#!/bin/bash

# string.bash --- bash emulation of string(3) library routines
# Author: Noah Friedman <friedman@prep.ai.mit.edu>
# ==>      Used with his kind permission in this document.
# Created: 1992-07-01
# Last modified: 1993-09-29
# Public domain

# Conversion to bash v2 syntax done by Chet Ramey

# Commentary:
# Code:

#:docstring strcat:
# Usage: strcat s1 s2
#
# Strcat appends the value of variable s2 to variable s1.
#
# Example:
#   a="foo"
#   b="bar"
#   strcat a b
#   echo $a
#   => foobar
#
#:end docstring:

###;;;autoload    ==> Autoloading of function commented out.
function strcat ()
{

```

```

    local s1_val s2_val

    s1_val=${!1}                # indirect variable expansion
    s2_val=${!2}
    eval "$1"="\${s1_val}${s2_val}"\
    # ==> eval $1='${s1_val}${s2_val}' avoids problems,
    # ==> if one of the variables contains a single quote.
}

#:docstring strncat:
# Usage: strncat s1 s2 $n
#
# Line strcat, but strncat appends a maximum of n characters from the value
# of variable s2. It copies fewer if the value of variable s2 is shorter
# than n characters. Echoes result on stdout.
#
# Example:
#   a=foo
#   b=barbaz
#   strncat a b 3
#   echo $a
#   => foobar
#
#:end docstring:

###;;autoload
function strncat ()
{
    local s1="$1"
    local s2="$2"
    local -i n="$3"
    local s1_val s2_val

    s1_val=${!s1}                # ==> indirect variable expansion
    s2_val=${!s2}

    if [ ${#s2_val} -gt $n ]; then
        s2_val=${s2_val:0:$n}    # ==> substring extraction
    fi

    eval "$s1"="\${s1_val}${s2_val}"\
    # ==> eval $1='${s1_val}${s2_val}' avoids problems,
    # ==> if one of the variables contains a single quote.
}

#:docstring strcmp:
# Usage: strcmp $s1 $s2
#
# Strcmp compares its arguments and returns an integer less than, equal to,
# or greater than zero, depending on whether string s1 is lexicographically
# less than, equal to, or greater than string s2.
#:end docstring:

###;;autoload
function strcmp ()
{

```

```

    [ "$1" = "$2" ] && return 0

    [ "${1}" '<' "${2}" ] > /dev/null && return -1

    return 1
}

#:docstring strncmp:
# Usage: strncmp $s1 $s2 $n
#
# Like strcmp, but makes the comparison by examining a maximum of n
# characters (n less than or equal to zero yields equality).
#:end docstring:

####;;autoload
function strncmp ()
{
    if [ -z "${3}" -o "${3}" -le "0" ]; then
        return 0
    fi

    if [ ${3} -ge ${#1} -a ${3} -ge ${#2} ]; then
        strcmp "$1" "$2"
        return $?
    else
        s1=${1:0:$3}
        s2=${2:0:$3}
        strcmp $s1 $s2
        return $?
    fi
}

#:docstring strlen:
# Usage: strlen s
#
# Strlen returns the number of characters in string literal s.
#:end docstring:

####;;autoload
function strlen ()
{
    eval echo "\${#${1}}"
    # ==> Returns the length of the value of the variable
    # ==> whose name is passed as an argument.
}

#:docstring strstrpn:
# Usage: strstrpn $s1 $s2
#
# Strspn returns the length of the maximum initial segment of string s1,
# which consists entirely of characters from string s2.
#:end docstring:

####;;autoload
function strstrpn ()
{

```

```

    # Unsetting IFS allows whitespace to be handled as normal chars.
    local IFS=
    local result="${1%%[!${2}]*}"

    echo ${#result}
}

#:docstring strcspn:
# Usage: strcspn $s1 $s2
#
# Strcspn returns the length of the maximum initial segment of string s1,
# which consists entirely of characters not from string s2.
#:end docstring:

###;;;autoload
function strcspn ()
{
    # Unsetting IFS allows whitespace to be handled as normal chars.
    local IFS=
    local result="${1%%[!${2}]*}"

    echo ${#result}
}

#:docstring strstr:
# Usage: strstr s1 s2
#
# Strstr echoes a substring starting at the first occurrence of string s2 in
# string s1, or nothing if s2 does not occur in the string. If s2 points to
# a string of zero length, strstr echoes s1.
#:end docstring:

###;;;autoload
function strstr ()
{
    # if s2 points to a string of zero length, strstr echoes s1
    [ ${#2} -eq 0 ] && { echo "$1" ; return 0; }

    # strstr echoes nothing if s2 does not occur in s1
    case "$1" in
    *$2*) ;;
    *) return 1;;
    esac

    # use the pattern matching code to strip off the match and everything
    # following it
    first=${1/$2*/}

    # then strip off the first unmatched portion of the string
    echo "${1##$first}"
}

#:docstring strtok:
# Usage: strtok s1 s2
#
# Strtok considers the string s1 to consist of a sequence of zero or more

```



```

# text tokens separated by spans of one or more characters from the
# separator string s2. The first call (with a non-empty string s1
# specified) echoes a string consisting of the first token on stdout. The
# function keeps track of its position in the string s1 between separate
# calls, so that subsequent calls made with the first argument an empty
# string will work through the string immediately following that token. In
# this way subsequent calls will work through the string s1 until no tokens
# remain. The separator string s2 may be different from call to call.
# When no token remains in s1, an empty value is echoed on stdout.
#:end docstring:

###;;;autoload
function strtok ()
{
:
}

#:docstring strtrunc:
# Usage: strtrunc $n $s1 {$s2} {$...}
#
# Used by many functions like strncmp to truncate arguments for comparison.
# Echoes the first n characters of each string s1 s2 ... on stdout.
#:end docstring:

###;;;autoload
function strtrunc ()
{
n=$1 ; shift
for z; do
echo "${z:0:$n}"
done
}

# provide string

# string.bash ends here

# =====
#
# ==> Everything below here added by the document author.

# ==> Suggested use of this script is to delete everything below here,
# ==> and "source" this file into your own scripts.

# strcat
string0=one
string1=two
echo
echo "Testing \"strcat\" function:"
echo "Original \"string0\" = $string0"
echo "\"string1\" = $string1"
strcat string0 string1
echo "New \"string0\" = $string0"
echo

```

```

# strlen
echo
echo "Testing \"strlen\" function:"
str=123456789
echo "\"str\" = $str"
echo -n "Length of \"str\" = "
strlen str
echo

# Exercise:
# -----
# Add code to test all the other string functions above.

exit 0

```

Michael Zick's complex array example uses the [md5sum](#) check sum command to encode directory information.

Example A-19. Directory information

```

#!/bin/bash
# directory-info.sh
# Parses and lists directory information.

# NOTE: Change lines 273 and 353 per "README" file.

# Michael Zick is the author of this script.
# Used here with his permission.

# Controls
# If overridden by command arguments, they must be in the order:
#   Arg1: "Descriptor Directory"
#   Arg2: "Exclude Paths"
#   Arg3: "Exclude Directories"
#
# Environment Settings override Defaults.
# Command arguments override Environment Settings.

# Default location for content addressed file descriptors.
MD5UCFS=${1:-${MD5UCFS:-'/tmpfs/ucfs'}}

# Directory paths never to list or enter
declare -a \
    EXCLUDE_PATHS=${2:-${EXCLUDE_PATHS:-'(/proc /dev /devfs /tmpfs)'}}

# Directories never to list or enter
declare -a \
    EXCLUDE_DIRS=${3:-${EXCLUDE_DIRS:-'(ucfs lost+found tmp wtmp)'}}

# Files never to list or enter
declare -a \

```

```

EXCLUDE_FILES=${3:-${EXCLUDE_FILES:-'(core "Name with Spaces")'}}

# Here document used as a comment block.
: <<LSfieldsDoc
# # # # # List Filesystem Directory Information # # # # #
#
#     ListDirectory "FileGlob" "Field-Array-Name"
# or
#     ListDirectory -of "FileGlob" "Field-Array-Filename"
#     '-of' meaning 'output to filename'
# # # # #

String format description based on: ls (GNU fileutils) version 4.0.36

Produces a line (or more) formatted:
inode permissions hard-links owner group ...
32736 -rw-----    1 mszick  mszick

size    day month date hh:mm:ss year path
2756608 Sun Apr 20 08:53:06 2003 /home/mszick/core

Unless it is formatted:
inode permissions hard-links owner group ...
266705 crw-rw----    1    root  uucp

major minor day month date hh:mm:ss year path
4,  68 Sun Apr 20 09:27:33 2003 /dev/ttyS4
NOTE: that pesky comma after the major number

NOTE: the 'path' may be multiple fields:
/home/mszick/core
/proc/982/fd/0 -> /dev/null
/proc/982/fd/1 -> /home/mszick/.xsession-errors
/proc/982/fd/13 -> /tmp/tmpfZVVOcs (deleted)
/proc/982/fd/7 -> /tmp/kde-mszick/ksycoca
/proc/982/fd/8 -> socket:[11586]
/proc/982/fd/9 -> pipe:[11588]

If that isn't enough to keep your parser guessing,
either or both of the path components may be relative:
../Built-Shared -> Built-Static
../linux-2.4.20.tar.bz2 -> ../../../../SRCS/linux-2.4.20.tar.bz2

The first character of the 11 (10?) character permissions field:
's' Socket
'd' Directory
'b' Block device
'c' Character device
'l' Symbolic link
NOTE: Hard links not marked - test for identical inode numbers
on identical filesystems.
All information about hard linked files are shared, except
for the names and the name's location in the directory system.
NOTE: A "Hard link" is known as a "File Alias" on some systems.
'-' An undistinguished file

```

Followed by three groups of letters for: User, Group, Others

Character 1: '-' Not readable; 'r' Readable

Character 2: '-' Not writable; 'w' Writable

Character 3, User and Group: Combined execute and special

'-' Not Executable, Not Special

'x' Executable, Not Special

's' Executable, Special

'S' Not Executable, Special

Character 3, Others: Combined execute and sticky (tacky?)

'-' Not Executable, Not Tacky

'x' Executable, Not Tacky

't' Executable, Tacky

'T' Not Executable, Tacky

Followed by an access indicator

Haven't tested this one, it may be the eleventh character

or it may generate another field

' ' No alternate access

'+' Alternate access

LSfieldsDoc

ListDirectory()

```
{
    local -a T
    local -i of=0          # Default return in variable
#    OLD_IFS=$IFS          # Using BASH default ' \t\n'

    case "$#" in
        3) case "$1" in
            -of) of=1 ; shift ;;
            * ) return 1 ;;
            esac ;;
        2) : ;;           # Poor man's "continue"
        *) return 1 ;;
    esac

    # NOTE: the (ls) command is NOT quoted (")
    T=( $(ls --inode --ignore-backups --almost-all --directory \
--full-time --color=none --time=status --sort=none \
--format=long $1) )

    case $of in
        # Assign T back to the array whose name was passed as $2
        0) eval $2=\( \ "${T[@]}" \) ;;
        # Write T into filename passed as $2
        1) echo "${T[@]}" > "$2" ;;
    esac
    return 0
}

# # # # # Is that string a legal number? # # # # #
#
#     IsNumber "Var"
# # # # # There has to be a better way, sigh...
```

```

IsNumber()
{
    local -i int
    if [ $# -eq 0 ]
    then
        return 1
    else
        (let int=$1) 2>/dev/null
        return $?      # Exit status of the let thread
    fi
}

# # # # # Index Filesystem Directory Information # # # # #
#
#       IndexList "Field-Array-Name" "Index-Array-Name"
# or
#       IndexList -if Field-Array-Filename Index-Array-Name
#       IndexList -of Field-Array-Name Index-Array-Filename
#       IndexList -if -of Field-Array-Filename Index-Array-Filename
# # # # #

: <<IndexListDoc
Walk an array of directory fields produced by ListDirectory

Having suppressed the line breaks in an otherwise line oriented
report, build an index to the array element which starts each line.

Each line gets two index entries, the first element of each line
(inode) and the element that holds the pathname of the file.

The first index entry pair (Line-Number==0) are informational:
Index-Array-Name[0] : Number of "Lines" indexed
Index-Array-Name[1] : "Current Line" pointer into Index-Array-Name

The following index pairs (if any) hold element indexes into
the Field-Array-Name per:
Index-Array-Name[Line-Number * 2] : The "inode" field element.
NOTE: This distance may be either +11 or +12 elements.
Index-Array-Name[(Line-Number * 2) + 1] : The "pathname" element.
NOTE: This distance may be a variable number of elements.
Next line index pair for Line-Number+1.
IndexListDoc

IndexList()
{
    local -a LIST                # Local of listname passed
    local -a -i INDEX=( 0 0 )   # Local of index to return
    local -i Lidx Lcnt
    local -i if=0 of=0          # Default to variable names

    case "$#" in
        0) return 1 ;;
        1) return 1 ;;

```

```

2) : ;;                                # Poor man's continue
3) case "$1" in
    -if) if=1 ;;
    -of) of=1 ;;
    * ) return 1 ;;
    esac ; shift ;;
4) if=1 ; of=1 ; shift ; shift ;;
*) return 1

esac

# Make local copy of list
case "$if" in
0) eval LIST=\( \("${$1[@]}\)" \) ;;
1) LIST=( $(cat $1) ) ;;
esac

# Grok (grope?) the array
Lcnt=${#LIST[@]}
Lidx=0
until (( Lidx >= Lcnt ))
do
if IsNumber ${LIST[$Lidx]}
then
    local -i inode name
    local ft
    inode=Lidx
    local m=${LIST[$Lidx+2]}          # Hard Links field
    ft=${LIST[$Lidx+1]:0:1}          # Fast-Stat
    case $ft in
    b)      ((Lidx+=12)) ;;           # Block device
    c)      ((Lidx+=12)) ;;           # Character device
    *)      ((Lidx+=11)) ;;           # Anything else
    esac
    name=Lidx
    case $ft in
    -)      ((Lidx+=1)) ;;           # The easy one
    b)      ((Lidx+=1)) ;;           # Block device
    c)      ((Lidx+=1)) ;;           # Character device
    d)      ((Lidx+=1)) ;;           # The other easy one
    l)      ((Lidx+=3)) ;;           # At LEAST two more fields
    esac
    # A little more elegance here would handle pipes,
    #+ sockets, deleted files - later.
    *)      until IsNumber ${LIST[$Lidx]} || ((Lidx >= Lcnt))
            do
                ((Lidx+=1))
            done
            ;;
    esac
    INDEX[${#INDEX[*]}]=$inode
    INDEX[${#INDEX[*]}]=$name
    INDEX[0]=${INDEX[0]}+1          # One more "line" found
# echo "Line: ${INDEX[0]} Type: $ft Links: $m Inode: \
# ${LIST[$inode]} Name: ${LIST[$name]}"

else
    ((Lidx+=1))

```

```

        fi
    done
    case "$of" in
        0) eval $2=\( \("${INDEX[@]}\\" \) ;;
        1) echo "${INDEX[@]}" > "$2" ;;
    esac
    return 0                                # What could go wrong?
}

# # # # # Content Identify File # # # # #
#
#       DigestFile Input-Array-Name Digest-Array-Name
# or
#       DigestFile -if Input-FileName Digest-Array-Name
# # # # #

# Here document used as a comment block.
: <<DigestFilesDoc

The key (no pun intended) to a Unified Content File System (UCFS)
is to distinguish the files in the system based on their content.
Distinguishing files by their name is just so 20th Century.

The content is distinguished by computing a checksum of that content.
This version uses the md5sum program to generate a 128 bit checksum
representative of the file's contents.
There is a chance that two files having different content might
generate the same checksum using md5sum (or any checksum). Should
that become a problem, then the use of md5sum can be replace by a
cryptographic signature. But until then...

The md5sum program is documented as outputting three fields (and it
does), but when read it appears as two fields (array elements). This
is caused by the lack of whitespace between the second and third field.
So this function gropes the md5sum output and returns:
    [0]      32 character checksum in hexadecimal (UCFS filename)
    [1]      Single character: ' ' text file, '*' binary file
    [2]      Filesystem (20th Century Style) name
    Note: That name may be the character '-' indicating STDIN read.

DigestFilesDoc

DigestFile()
{
    local if=0                                # Default, variable name
    local -a T1 T2

    case "$#" in
        3) case "$1" in
            -if) if=1 ; shift ;;
              * ) return 1 ;;
            esac ;;
        2) : ;;                                # Poor man's "continue"
        *) return 1 ;;
    esac
}

```

```

esac

case $if in
0) eval T1=\( \"\$\{$1\[@\]\}\\" \)
    T2=( $(echo ${T1[@]} | md5sum -) )
    ;;
1) T2=( $(md5sum $1) )
    ;;
esac

case ${#T2[@]} in
0) return 1 ;;
1) return 1 ;;
2) case ${T2[1]:0:1} in
        \*) T2[${#T2[@]}]=${T2[1]:1}          # SanScrit-2.0.5
            T2[1]=\*
            ;;
        *) T2[${#T2[@]}]=${T2[1]}
            T2[1]=" "
            ;;
    esac
    ;;
3) : ;; # Assume it worked
*) return 1 ;;
esac

local -i len=${#T2[0]}
if [ $len -ne 32 ] ; then return 1 ; fi
eval $2=\( \"\$\{T2\[@\]\}\\" \)
}

# # # # # Locate File # # # # #
#
#       LocateFile [-l] FileName Location-Array-Name
# or
#       LocateFile [-l] -of FileName Location-Array-FileName
# # # # #

# A file location is Filesystem-id and inode-number

# Here document used as a comment block.
: <<StatFieldsDoc
Based on stat, version 2.2
stat -t and stat -lt fields
[0]      name
[1]      Total size
          File - number of bytes
          Symbolic link - string length of pathname
[2]      Number of (512 byte) blocks allocated
[3]      File type and Access rights (hex)
[4]      User ID of owner
[5]      Group ID of owner
[6]      Device number
[7]      Inode number
[8]      Number of hard links
[9]      Device type (if inode device) Major

```



```

[10] Device type (if inode device) Minor
[11] Time of last access
    May be disabled in 'mount' with noatime
    atime of files changed by exec, read, pipe, utime, mknod
(mmap?)
    atime of directories changed by addition/deletion of files
[12] Time of last modification
    mtime of files changed by write, truncate, utime, mknod
    mtime of directories changed by addition/deletion of files
[13] Time of last change
    ctime reflects time of changed inode information (owner,
group
    permissions, link count

```

*** Per:

```

Return code: 0
Size of array: 14
Contents of array
Element 0: /home/mszick
Element 1: 4096
Element 2: 8
Element 3: 41e8
Element 4: 500
Element 5: 500
Element 6: 303
Element 7: 32385
Element 8: 22
Element 9: 0
Element 10: 0
Element 11: 1051221030
Element 12: 1051214068
Element 13: 1051214068

```

For a link in the form of linkname -> realname
stat -t linkname returns the linkname (link) information
stat -lt linkname returns the realname information

stat -tf and stat -ltf fields

```

[0] name
[1] ID-0? # Maybe someday, but Linux stat structure
[2] ID-0? # does not have either LABEL nor UUID
        # fields, currently information must come
        # from file-system specific utilities

```

These will be munged into:

```

[1] UUID if possible
[2] Volume Label if possible

```

Note: 'mount -l' does return the label and could return the UUID

```

[3] Maximum length of filenames
[4] Filesystem type
[5] Total blocks in the filesystem
[6] Free blocks
[7] Free blocks for non-root user(s)
[8] Block size of the filesystem
[9] Total inodes
[10] Free inodes

```

```

-*- Per:
    Return code: 0
    Size of array: 11
    Contents of array
    Element 0: /home/mszick
    Element 1: 0
    Element 2: 0
    Element 3: 255
    Element 4: ef53
    Element 5: 2581445
    Element 6: 2277180
    Element 7: 2146050
    Element 8: 4096
    Element 9: 1311552
    Element 10: 1276425

StatFieldsDoc

#      LocateFile [-l] FileName Location-Array-Name
#      LocateFile [-l] -of FileName Location-Array-FileName

LocateFile()
{
    local -a LOC LOC1 LOC2
    local lk="" of=0

    case "$#" in
        0) return 1 ;;
        1) return 1 ;;
        2) : ;;
        *) while (( "$#" > 2 ))
            do
                case "$1" in
                    -l) lk=-1 ;;
                    -of) of=1 ;;
                    *) return 1 ;;
                esac
                shift
            done ;;
    esac

# More Sanscrit-2.0.5
# LOC1=( $(stat -t $lk $1) )
# LOC2=( $(stat -tf $lk $1) )
# Uncomment above two lines if system has "stat" command installed.
LOC=( ${LOC1[@]:0:1} ${LOC1[@]:3:11}
      ${LOC2[@]:1:2} ${LOC2[@]:4:1} )

    case "$of" in
        0) eval $2=\( \`${LOC[@]}\` \) ;;
        1) echo "${LOC[@]}" > "$2" ;;
    esac
    return 0
# Which yields (if you are lucky, and have "stat" installed)
# -*- Location Discriptor -*-

```

```

#      Return code: 0
#      Size of array: 15
#      Contents of array
#      Element 0: /home/mszick          20th Century name
#      Element 1: 41e8                  Type and Permissions
#      Element 2: 500                    User
#      Element 3: 500                    Group
#      Element 4: 303                    Device
#      Element 5: 32385                  inode
#      Element 6: 22                     Link count
#      Element 7: 0                      Device Major
#      Element 8: 0                      Device Minor
#      Element 9: 1051224608             Last Access
#      Element 10: 1051214068            Last Modify
#      Element 11: 1051214068            Last Status
#      Element 12: 0                     UUID (to be)
#      Element 13: 0                     Volume Label (to be)
#      Element 14: ef53                  Filesystem type
}

```

```

# And then there was some test code

```

```

ListArray() # ListArray Name

```

```

{
    local -a Ta

    eval Ta=( \ "${1%[@]} \ " \ )
    echo
    echo "--*- List of Array *-*- "
    echo "Size of array $1: ${#Ta[*]}"
    echo "Contents of array $1:"
    for (( i=0 ; i<${#Ta[*]} ; i++ ))
    do
        echo -e "\tElement $i: ${Ta[$i]}"
    done
    return 0
}

```

```

declare -a CUR_DIR
# For small arrays
ListDirectory "${PWD}" CUR_DIR
ListArray CUR_DIR

```

```

declare -a DIR_DIG
DigestFile CUR_DIR DIR_DIG
echo "The new \"name\" (checksum) for ${CUR_DIR[9]} is ${DIR_DIG[0]}"

```

```

declare -a DIR_ENT
# BIG_DIR # For really big arrays - use a temporary file in ramdisk
# BIG-DIR # ListDirectory -of "${CUR_DIR[11]}/*" "/tmpfs/junk2"
ListDirectory "${CUR_DIR[11]}/*" DIR_ENT

```

```

declare -a DIR_IDX
# BIG-DIR # IndexList -if "/tmpfs/junk2" DIR_IDX

```

```

IndexList DIR_ENT DIR_IDX

declare -a IDX_DIG
# BIG-DIR # DIR_ENT=( $(cat /tmpfs/junk2) )
# BIG-DIR # DigestFile -if /tmpfs/junk2 IDX_DIG
DigestFile DIR_ENT IDX_DIG
# Small (should) be able to parallize IndexList & DigestFile
# Large (should) be able to parallize IndexList & DigestFile & the assignment
echo "The \"name\" (checksum) for the contents of ${PWD} is ${IDX_DIG[0]}"

declare -a FILE_LOC
LocateFile ${PWD} FILE_LOC
ListArray FILE_LOC

exit 0

```

Stéphane Chazelas demonstrates object-oriented programming in a Bash script.

Mariusz Gniazdowski contributed a [hash](#) library for use in scripts.

Example A-20. Library of hash functions

```

# Hash:
# Hash function library
# Author: Mariusz Gniazdowski <mariusz.gn-at-gmail.com>
# Date: 2005-04-07

# Functions making emulating hashes in Bash a little less painful.

#   Limitations:
#   * Only global variables are supported.
#   * Each hash instance generates one global variable per value.
#   * Variable names collisions are possible
#+  if you define variable like __hash__hashname_key
#   * Keys must use chars that can be part of a Bash variable name
#+  (no dashes, periods, etc.).
#   * The hash is created as a variable:
#     ... hashname_keyname
#     So if someone will create hashes like:
#       myhash_ + mykey = myhash__mykey
#       myhash + _mykey = myhash__mykey
#     Then there will be a collision.
#     (This should not pose a major problem.)

Hash_config_varname_prefix=__hash__

# Emulates: hash[key]=value
#
# Params:
# 1 - hash
# 2 - key

```

```

# 3 - value
function hash_set {
    eval "${Hash_config_varname_prefix}${1}_${2}=\"${3}\""
}

# Emulates:  value=hash[key]
#
# Params:
# 1 - hash
# 2 - key
# 3 - value (name of global variable to set)
function hash_get_into {
    eval "$3=\"\${Hash_config_varname_prefix}${1}_${2}\""
}

# Emulates:  echo hash[key]
#
# Params:
# 1 - hash
# 2 - key
# 3 - echo params (like -n, for example)
function hash_echo {
    eval "echo $3 \"\${Hash_config_varname_prefix}${1}_${2}\""
}

# Emulates:  hash1[key1]=hash2[key2]
#
# Params:
# 1 - hash1
# 2 - key1
# 3 - hash2
# 4 - key2
function hash_copy {
    eval "${Hash_config_varname_prefix}${1}_${2}\
=\"\${Hash_config_varname_prefix}${3}_${4}\""
}

# Emulates:  hash[keyN-1]=hash[key2]=...hash[key1]
#
# Copies first key to rest of keys.
#
# Params:
# 1 - hash1
# 2 - key1
# 3 - key2
# . . .
# N - keyN
function hash_dup {
    local hashName="$1" keyName="$2"
    shift 2
    until [ $# -le 0 ]; do
        eval "${Hash_config_varname_prefix}${hashName}_${1}\

```

```

=\\"\\${Hash_config_varname_prefix}${hashName}_${keyName}\\\"
    shift;
    done;
}

# Emulates:  unset hash[key]
#
# Params:
# 1 - hash
# 2 - key
function hash_unset {
    eval "unset ${Hash_config_varname_prefix}${1}_${2}"
}

# Emulates something similar to:  ref=&hash[key]
#
# The reference is name of the variable in which value is held.
#
# Params:
# 1 - hash
# 2 - key
# 3 - ref - Name of global variable to set.
function hash_get_ref_into {
    eval "$3=\"${Hash_config_varname_prefix}${1}_${2}\""
}

# Emulates something similar to:  echo &hash[key]
#
# That reference is name of variable in which value is held.
#
# Params:
# 1 - hash
# 2 - key
# 3 - echo params (like -n for example)
function hash_echo_ref {
    eval "echo $3 \"${Hash_config_varname_prefix}${1}_${2}\""
}

# Emulates something similar to:  $$hash[key](param1, param2, ...)
#
# Params:
# 1 - hash
# 2 - key
# 3,4, ... - Function parameters
function hash_call {
    local hash key
    hash=$1
    key=$2
    shift 2
    eval "eval \"\\\"\\${Hash_config_varname_prefix}${hash}_${key} \\\"\\\"\\\"$@\\\"\\\"\""
}

```

```

# Emulates something similar to:  isset(hash[key]) or hash[key]==NULL
#
# Params:
# 1 - hash
# 2 - key
# Returns:
# 0 - there is such key
# 1 - there is no such key
function hash_is_set {
    eval "if [[ \"\${${Hash_config_varname_prefix}${1}_${2}-a}\" = \"a\" &&
    \"\${${Hash_config_varname_prefix}${1}_${2}-b}\" = \"b\" ]]
    then return 1; else return 0; fi"
}

# Emulates something similar to:
#   foreach($hash as $key => $value) { fun($key,$value); }
#
# It is possible to write different variations of this function.
# Here we use a function call to make it as "generic" as possible.
#
# Params:
# 1 - hash
# 2 - function name
function hash_foreach {
    local keyname oldIFS="$IFS"
    IFS=' '
    for i in $(eval "echo \${!${Hash_config_varname_prefix}${1}_*}"); do
        keyname=$(eval "echo \${i##${Hash_config_varname_prefix}${1}_}")
        eval "$2 $keyname \"\${$i}\""
    done
    IFS="$oldIFS"
}

# NOTE: In lines 103 and 116, ampersand changed.
# But, it doesn't matter, because these are comment lines anyhow

```

2) Analyzing Scripts

Provide, in the same way as above a complete analysis examining the following scripts.

- a) Run the following script, then explain what it does. Annotate the script and rewrite it in a more compact and elegant manner.

```

#!/bin/bash

MAX=10000

for((nr=1; nr<$MAX; nr++))

```

```

do

    let "t1 = nr % 5"
    if [ "$t1" -ne 3 ]
    then
        continue
    fi

    let "t2 = nr % 7"
    if [ "$t2" -ne 4 ]
    then
        continue
    fi

    let "t3 = nr % 9"
    if [ "$t3" -ne 5 ]
    then
        continue
    fi

    break # What happens when you comment out this line? Why?

done

echo "Number = $nr"

exit 0

```

- b) Explain what the following script does. It is really just a parameterized command-line pipe.

```

#!/bin/bash

DIRNAME=/usr/bin
FILETYPE="shell script"
LOGFILE=logfile

file "$DIRNAME"/* | fgrep "$FILETYPE" | tee $LOGFILE | wc -l

exit 0

```

- c) Examine and explain the following script. For hints, you might refer to the listings for [find](#) and [stat](#).


```
#!/bin/bash

# Author: Nathan Coulter
# This code is released to the public domain.
# The author gave permission to use this code snippet in the ABS Guide.

find -maxdepth 1 -type f -printf '%f\000' | {
  while read -d $'\000'; do
    mv "$REPLY" "$(date -d "$(stat -c '%y' "$REPLY")" "+%Y%m%d%H%M%S"
    )-$REPLY"
  done
}

# Warning: Test-drive this script in a "scratch" directory.
# It will somehow affect all the files there.
```

d) A reader sent in the following code snippet.

```
while read LINE
do
  echo $LINE
done < `tail -f /var/log/messages`
```

He wished to write a script tracking changes to the system log file, /var/log/messages. Unfortunately, the above code block hangs and does nothing useful. Why? Fix this so it does work. (Hint: rather than [redirecting the stdin of the loop](#), try a [pipe](#).)

e) Analyze the following "one-liner" (here split into two lines for clarity) contributed by Rory Winston:

```
export SUM=0; for f in $(find src -name "*.java");
do export SUM=$((SUM + $(wc -l $f | awk '{ print $1 }'))); done; echo $SUM
```

Hint: First, break the script up into bite-sized sections. Then, carefully examine its use of [double-parentheses](#) arithmetic, the [export](#) command, the [find](#) command, the [wc](#) command, and [awk](#).

3) Designing Scripts

1. Suppose in a directory that you wanted to change all the filenames ending in .f77 so that they instead ended in .f90. How would you go about it?

If you're new to Unix you may need to find out how to change filenames. Typing `apropos filename` will list the programs and routines whose summaries mention `filename`. Alas, the most useful command isn't mentioned there. Typing `apropos rename` lists `mv` which is what you need. Maybe next you might try `"mv *.f77 *.f90"`. Alas, this won't work - the shell will replace `*.f77` by a list of filenames and the resulting command will fail. You need to use a loop of some sort. You also need a way to remove a suffix (look up `basename`) and how to add a new suffix. It's worth experimenting with a single name first. Do `filename=test.f77` and see if you can produce a `test.f90` string from `$filename`. One solution is

```
for filename in *.f77
do
    b=$(basename $filename .f77)
    mv $filename $b.f90
done
```

2. Change the `args` script supplied earlier so that if no argument is provided, "They are" isn't printed, and if exactly 1 argument is provided, "... 1 argument" rather than "... 1 arguments" is printed (use `if`)
3. Write a shell script that given a person's uid, tells you how many times that person is logged on. (`who`, `grep`, `wc`)
4. Write a shell script called `lsdirs` which lists *just* the directories in the current directory (test).
5. Write a shell script called `see` taking a filename name as argument which uses `ls` if the file's a directory and `more` if the file's otherwise (test)
6. Write a shell script that asks the user to type a word in, then tells the user how long that word is. (`read`, `wc`)
7. In many versions of unix there is a `-i` argument for `cp` so that you will be prompted for confirmation if you are about to overwrite a file. Write a script called `cpi` which will prompt if necessary without using the `-i` argument. (test)
8. Write a shell script that takes a uid as an argument and prints out that person's name, home directory, shell and group number. Print out the name of the group corresponding to the group number, and other groups that person may belong to. (`groups`, `awk`, `cut`. Also look at `/etc/passwd` and `/etc/groups`).
9. Sort `/etc/passwd` using the uid (first field) as the key. (sort)
10. Suppose that you want to write the same letter to many people except that you want each letter addressed to the senders personally. This *mailmerge* facility can be created using a shell script. Put the names of the recipients (one per line) in a file called `names`, create a textfile called `template` which has `NAME` wherever you want the person's name to appear and write a script (using `sed`) to produce a temporary file called `letter` from the `template` file.

11. Write a shell script that implements the Rubik Cube based encryption. See the details in <https://duksctf.github.io/2017/06/23/GoogleCTF2017-Rubik.html>

Part II

1) Implement

- A) **Multithreaded – synchronized solution** of the producer-consumer WITH BOUNDED BUFFER problem in POSIX PTHREADS and JAVA (or PYTHON). You could start from source codes at <https://macboypro.wordpress.com/2009/05/25/producer-consumer-problem-using-cpthreadsbounded-buffer/> , <https://medium.com/@basecs101/producer-consumer-problem-in-java-multi-threading-latest-2a306f003973> .

Make an extensive comparison between the two solutions in terms of performance. Start with two threads, that is one producer and one consumer and generalize to M producers, K consumers and a buffer of N positions. Provide all graphs for the performance of these two solutions concerning execution of the multi threads involved regarding all resources needed (memory , CPU usage etc.) and time needed to execute the relevant multithreaded systems as functions of N, M, K.

- B) **Multithreaded – synchronized solution** of the dining philosophers' synchronization problem again both in JAVA (or PYTHON), starting with the code here <https://www.baeldung.com/java-dining-philosophers> and in POSIX pthreads as here, <https://sites.cs.ucsb.edu/~rich/class/cs170/notes/DiningPhil/index.html>, using the algorithms and solutions developed in the Dining Philosophers Testbed with pthreads in this latter reference. Use, also, the solution we analyzed in the lectures (page 7.12 of the transparencies involving the monitor **monitor DiningPhilosophers**

```
{  
    enum { THINKING; HUNGRY, EATING) state [5] ;.....)
```

Again, as in A above generalize for N philosophers. Then, compare in detail all solutions providing all graphs for the performance of these solutions concerning execution of the multi threads involved regarding all resources needed (memory , CPU usage etc.) and time needed to execute the relevant multithreaded systems as functions of N.

2) Find the Optimal TIME QUANTUM in Round-Robin CPU scheduling algorithm in a multilevel feedback queue.

- A) Consider N processes that are produced randomly with RANDOM Burst times (in the range 10 to 1000 time units) and arriving at random moments within a range of time 0 to M time units $\ll N$. For instance 1000 processes arrived within 0..100 time units. N and M should vary, and they are independent variables. Consider the known multilevel feedback queue below,



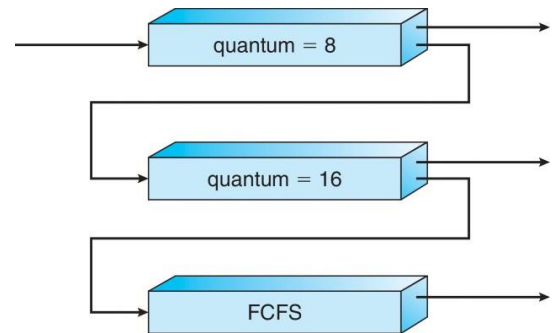
Example of Multilevel Feedback Queue

Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

Scheduling

- A new job enters queue Q_0 which is served FCFS
 - ▶ When it gains CPU, job receives 8 milliseconds
 - ▶ If it does not finish in 8 milliseconds, job is moved to queue Q_1
- At Q_1 job is again served FCFS and receives 16 additional milliseconds
 - ▶ If it still does not complete, it is preempted and moved to queue Q_2



Simulate this setup and study in detail through many runs (for instance 100-5000 processes for 100-500 independent simulations) the multilevel feedback queue scheduling problem in order to find optimal values for the two time quantum Q_1 , Q_2 above so as to achieve.

- n Max throughput
- n Min turnaround time
- n Min waiting time
- n Min response time

for all N processes.

Present all your results in detail in tables and graphs. The code should be in JAVA/C++ /or PYTHON (preferably in Python). Each level i of the three-level feedback queue above is assigned a time L_i . Therefore, if we assume 100 ms of time, the first level should run for L_1 , the second for L_2 and the third one for $100-L_1-L_2$. L_1 , L_2 are again independent variables. In conclusion, you should present graphs and tables of the above criteria

- n Max throughput
- n Min turnaround time
- n Min waiting time
- n Min response time

With respect to Q_1 , Q_2 , L_1 , L_2

- B) Consider the same setup as above but now in the third level both FCFS and SJF are running. Therefore, the time assigned for the third level is split in two using a Q percentage. Therefore, we run first SJF for T% of the time assigned in the third level and for (100-T)% we run second FCFS.

Present AGAIN all your results in detail in tables and graphs The code should be in JAVA/C++ /or PYTHON (preferably in Python). Each level i of the three level feedback queue above is assigned AGAIN a time L_i . Therefore, if we assume 100 ms of time the, the first level should run for L_1 , the second for L_2 and the third one for $100-L_1-L_2$. L_1 , L_2 are again independent variables. In conclusion you should present graphs and tables of the following criteria

- n Max throughput
- n Min turnaround time
- n Min waiting time
- n Min response time

With respect now to Q_1 , Q_2 , L_1 , L_2 T.

- 3) Consider again N processes that are produced randomly with RANDOM Burst times (in the range 10 to 1000 time units) and arriving at random moments within a range of time 0 to M time units For instance, 1000 processes arrived within 0..100 time units. N and M should vary, and they are independent variables. Suppose the same setup is repeated, after M time units have ended, with different N, M values again randomly selected. This process continues forever. Suppose finally that the system can SERVE up to R processes per time unit. The processes that exceed this number R should be postponed according to FCFS algorithm in order to be served. In this given setup calculate all quantities of M/M/1 and M/M/S queuing models. The population of processes is obviously infinite. You will calculate the averages of all known quantities when time goes to infinity. Also, calculate the same quantities when N, M, R are constant OVER TIME. In this case, draw all diagrams of how M/M/1, M/M/S quantities and probabilities vary with M, N, R. Is there any dependence on the random function used?? Change the random function used and show the differences.

For all projects you should provide all detailed reports required and all codes, programs and results. You submit one compressed file with the basic DETAILED MSWORD report including all results, designs, methodologies, implementations, codes, all simulation screenshots etc. and discussions as well as separately as different files all support files needed, including codes, screenshots runs, screenshots of performance analysis etc. Submission of a project without the full MS WORD report it is not taken into account.